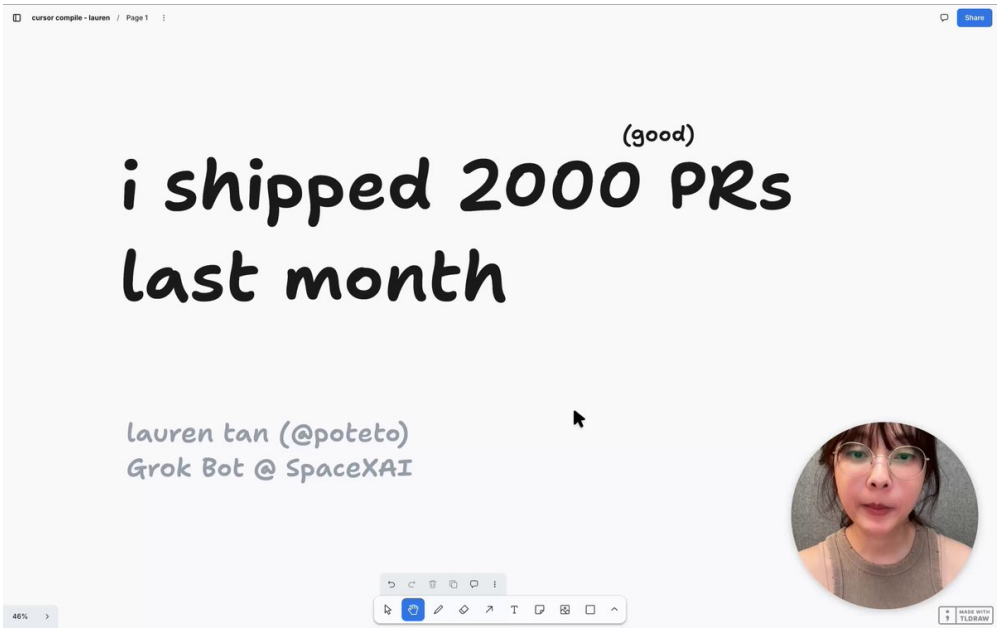
如何让代理可靠地交付代码：一间米其林厨房的工程设计

Lauren Tan 讲座整理：Codex

2026 年 9 月 29 日



视频作者/频道：Lauren Tan（@poteto）
发布日期：2026 年 9 月 21 日
视频时长：38 分 02 秒
视频链接：https://x.com/poteto/status/2102050467505430555

目录

1 为什么高产量首先是信任问题 **2**
1.1 从数字回到工作机制 . . . 2
1.2 性能问题怎样变成验证能力 . . . 3
1.3 本章小结 . . . 3

2 验证技能：给代理产品地图与可操作的手 **3**
2.1 可复用的 CLI 提供控制与证据 . . . 3
2.2 功能地图补上任务语境 . . . 3
2.3 经验验证与形式验证的边界 . . . 4
2.4 本章小结 . . . 4

3 把工程经验从人的脑中搬到环境里 **5**
3.1 正确性之外还有质量 . . . 5
3.2 每次纠错都问：哪一层能防止复发? . . . 5
3.3 代码库是会自我复制的记忆 . . . 5
3.4 本章小结 . . . 6

4 Dune：让正确路径成为默认路径 **6**
4.1 固定的概念和所有权 . . . 6
4.2 目录与导入规则体现进程边界 . . . 7
4.3 新增功能沿一条明确的垂直路径前进 . . . 8
4.4 本章小结 . . . 9

5 外部事件与内部代理组成自动化闭环 **9**
5.1 外层感知触发内层工作 . . . 9
5.2 本章小结 . . . 10

6 总结与延伸 **10**
6.1 讲者最后强调什么 . . . 10
6.2 把全场压缩成一个工程循环 . . . 10
6.3 本章小结 . . . 11

1 为什么高产量首先是信任问题

1.1 从数字回到工作机制

帖子文字称上个月交付了约 2500 个 PR；视频封面与开场则说约 2000 个。两个公开表述有差异，因而本讲义把数量视为讲者描述的量级，不用它推导精确产能。讲座真正解释的是：在人的注意力有限时，怎样让多个代理产出可以接受的代码。

讲者用“米其林厨房”比喻软件团队。厨师可以分工完成准备、烹饪和清洗，但主厨仍对出品负责。代理同样可以承担代码生成、复现和检查；人要设计工具、规则、架构与验收方式，使每个环节更可靠。她认为早期最困难的是从同时照看一到五个代理，过渡到能够放心并行更多任务。若每个代理都需要实时纠偏，数量越大，返工与回归越多。

核心命题

并行数量是信任的结果。先让代理能够理解任务、操作真实产品、拿到可复核的证据，再扩大同时运行的任务数。

图1复现讲者的概念曲线。它是定性示意，横轴上的代理数量与纵轴信任程度并非测量数据；曲线要表达的是，在初期建立信任时，每增加一位代理都可能明显增加监督负担。

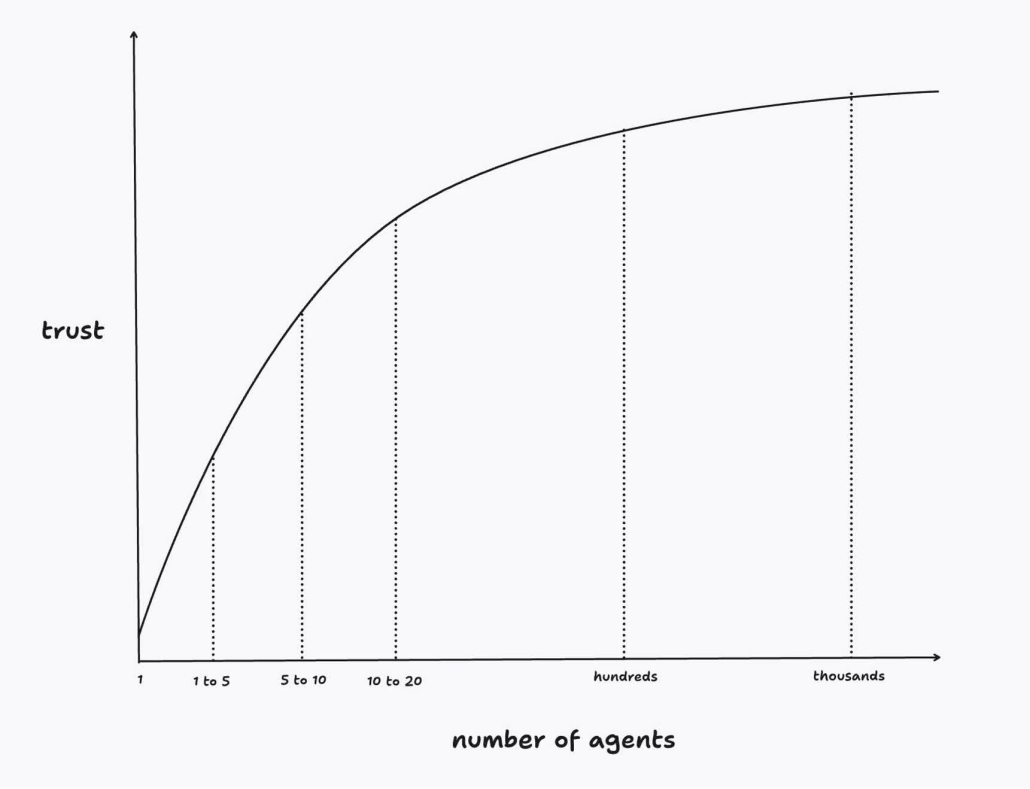


图 1: 信任与可并行代理数的定性关系。[1]

[1] 视频画面时间区间：00:06:20–00:06:40。

1.2 性能问题怎样变成验证能力

讲者刚加入 Cursor 时，负责改善新的代理窗口性能。团队的 PR 持续进入，人工打开 Chrome DevTools、录性能 trace、抓堆快照，难以持续监控新变更是否造成退化。这个经历促使她制作 Control Glass：教代理启动应用、采集性能证据、定位热点，反复修改并重新测量。这里的关键转变是把个人调试经验沉淀为代理可调用的验证流程。

三类证据

功能正确性需要真实操作与结果证据；性能需要 trace、耗时与内存等实测数据；代码质量还需要架构约束、静态检查和工程流程。单个“测试通过”无法覆盖所有维度。

1.3 本章小结

讲者的高并行实践从一个具体痛点开始：人工性能验收跟不上变更速度。可重复的验证能力使代理工作逐渐可信，之后并行才有意义。

2 验证技能：给代理产品地图与可操作的手

2.1 可复用的 CLI 提供控制与证据

一个代理每次临时写脚本启动应用，命令参数、环境准备和证据格式都可能漂移。讲者将稳定的 CLI 放进技能目录，让代理复用同一条路径：启动真实应用、进入目标界面、执行交互、抓取截图或 trace、报告结果。这样，人可以复核代理声称“修好了”的依据，也更容易比较修改前后。

2.2 功能地图补上任务语境

仅能操纵应用还不够。内部用户可能只发一小张截图和几个问号；代理虽能启动应用，却不知道截图对应哪个功能、怎样到达、期望行为是什么。讲者提出 feature map（功能地图）：记录产品有哪些功能、入口在哪里、用户怎样通过按钮或快捷键抵达，以及功能各自做什么。团队用自动化维护它。功能地图给语境，CLI 给控制，两者组合让代理能够从模糊反馈走向可执行的复现。

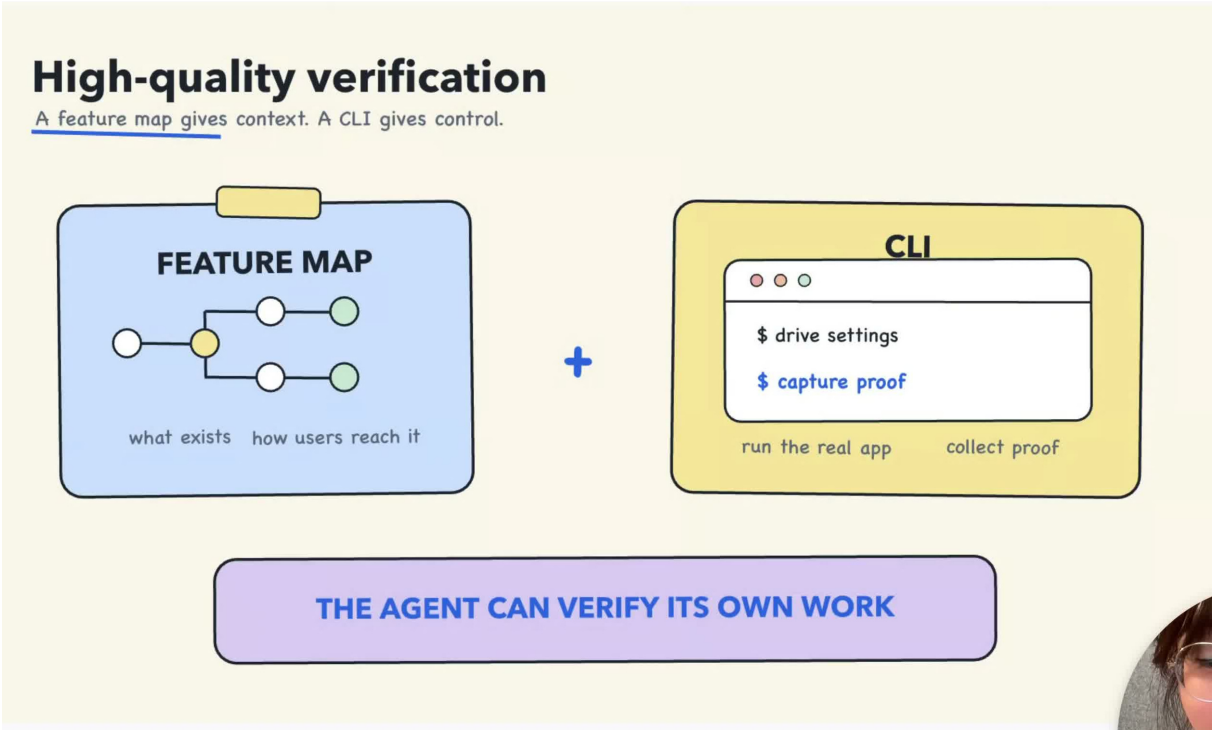


图 2: 功能地图负责“找对地方”，CLI 负责“运行并取证”。[2]

可操作的验收闭环

需求或故障报告 → 功能地图定位 → CLI 驱动真实应用 → 收集证据 → 修改 → 用同一流程复测。每一步都应能留下可供人检查的产物。

2.3 经验验证与形式验证的边界

讲者把验证看成一个范围。Control Glass 一类工具给出经验性证据，例如某条路径能成功运行、某次性能测量满足目标；形式方法如 Lean 或 TLA+ 则可能证明指定不变量。前者更容易落地，但只能支持已覆盖的场景；后者更强，也需要精确建模与更高投入。她没有声称截图或 trace 可以证明整个应用永远正确。

常见误读

代理“跑了一遍”只说明这次、这条路径观察到了相应结果。应明确运行环境、输入条件、断言和证据位置，避免把一次成功当作全局保证。

2.4 本章小结

验证技能不是一段提示词，而是产品语境、稳定控制接口和证据采集的组合。它让代理能够独立检查自己的工作，也让人的审核更有抓手。

[2] 视频画面时间区间：00:10:45–00:11:15。

3 把工程经验从人的脑中搬到环境里

3.1 正确性之外还有质量

讲者随后区分“功能做到了”与“代码做得好”。她以 pstack 插件中的调试、功能开发、原型设计等技能为例，把成熟工程师的工作方法写成可复用流程。性能指标也应来自真实测量，而非代码外观或自信的解释。技能能提高行为质量，但只有在任务中被正确调用时才起作用。

3.2 每次纠错都问：哪一层能防止复发？

如果代理反复犯同一个错误，逐次提醒会持续耗费注意力。讲者展示了五个层次：代码库结构；静态分析（lint、编译器、CI）；规则或自动审查；技能；风格指南和人工审阅。她建议优先考虑能更稳定执行的层次：修改数据结构或接口，使错误路径不可表达；其次是添加机器可执行的检查；再补上规则和技能。



图 3: 发现代理错误时，讲者建议检查的五层环境。[3]

例如，若代理不断从 Electron 主进程直接引入渲染进程不应运行的模块，单靠 style guide 要求“请小心”很脆弱。把目录、公开接口和导入方向限定清楚，再由 lint 或编译规则拦截越界导入，能把错误挡在合并前。规则文档仍有价值，可以解释原因与例外；它无需独自承担约束。

讲者的优先顺序

先看架构和数据结构能否让错误不可能发生；再用 lint、编译器与 CI 固化可检查条件；最后用规则、审查机器人和技能引导剩余判断。人工评论可作为发现缺口的信号。

3.3 代码库是会自我复制的记忆

代理读取现有代码，再延续它看到的模式。好模式因此降低新任务的判断成本；局部权宜做法也会被复制，几周后形成事实标准。讲者用“花园”比喻代码库：杂草越早处理，成本越低。对反复出现的反模式，先添加 lint 规则可以阻止扩散，再安排清理旧债。

[3] 视频画面时间区间：00:16:50–00:17:10。

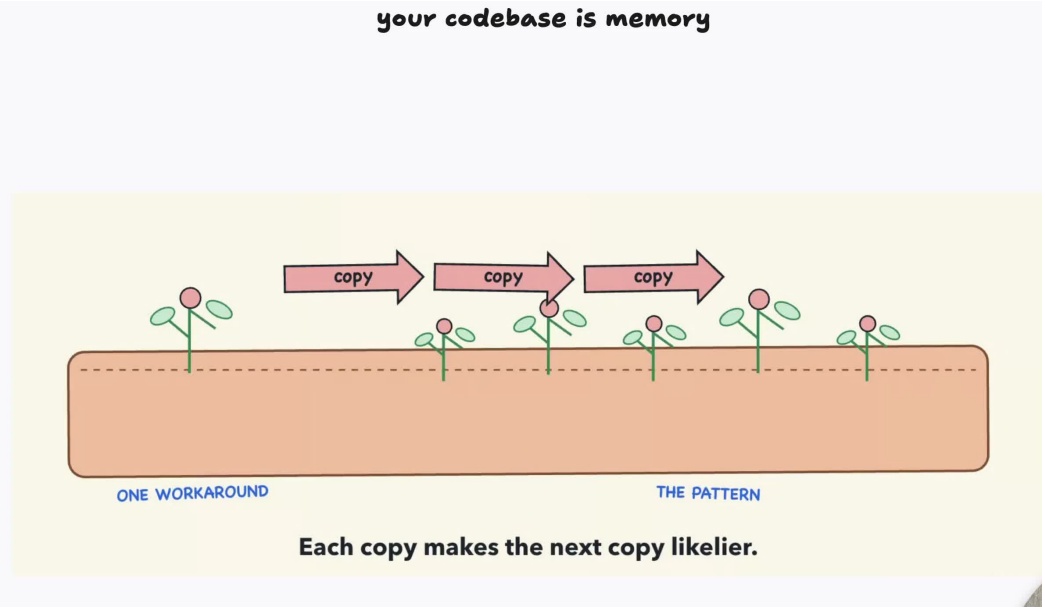


图 4: 一次变通被持续复制，逐渐变成默认模式。[4]

她举出的强硬例子是在 Dune 中禁止代码注释：观察到代理常把附近注释当作绕开根因、贴补丁的理由。这个做法属于该团队针对特定行为的选择，并非所有项目都应照搬。更普适的原则是识别实际被复制的坏模式，并用适合项目的约束处理它。

别把示例误写成通用规则

“禁注释”需要结合代码库与团队习惯评估。清晰解释设计原因的注释可能有价值；真正要拦截的是用注释为未解决的问题背书的行为。

3.4 本章小结

可扩展的纠错发生在环境里。一次 review 意见最好留下可复用的结构、检查或流程，让下一位代理更容易做对。

4 Dune：让正确路径成为默认路径

4.1 固定的概念和所有权

讲者介绍 Grokbot 客户端框架 Dune，重点不是推广某个框架，而是展示怎样把团队经验编码进项目骨架。幻灯片以五个名词组织应用：Feature 管理产品 UI 的归属目录；Entrypoint 是用户可打开的视图；Transcript card 表示一种内容卡片；Client 负责渲染侧持久状态、hooks 与命令；Host 在另一侧承载持续运行的行为。每个概念有明确的目录位置和运行职责。

[4] 视频画面时间区间：00:22:50–00:23:10。

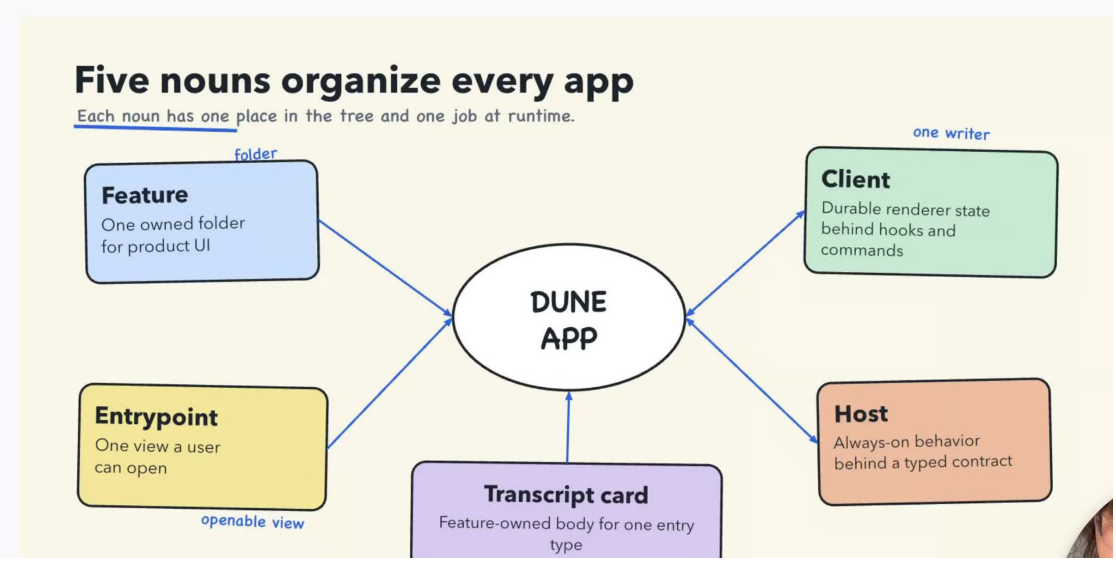


图 5: Dune 用五个固定概念组织应用。[5]

4.2 目录与导入规则体现进程边界

Electron 渲染线程要维持流畅响应；主进程或 Host 中昂贵的工作不应误入渲染线程。讲者提到 60 fps 时每帧约有 16.7 毫秒、120 fps 时约有 8.3 毫秒的预算。这只是帧间时间尺度，并不表示某个任务可以占满整个预算。Dune 通过目录和导入依赖图，把哪些代码在哪个进程运行、哪些跨边界引用合法，变成机器可检查的规则。

这些时间数字来自简单换算：一秒分为 f 帧，每帧平均时间为

$$T = \frac{1000}{f}\ \text{ms}.$$

- T：一帧对应的平均毫秒数。
- f：目标帧率，单位是每秒帧数。

因此 $f = 60$ 时约为 16.7 ms，$f = 120$ 时约为 8.3 ms；实际渲染还要为浏览器、绘制和其他工作留出余量。

[5] 视频画面时间区间：00:27:20–00:27:40。

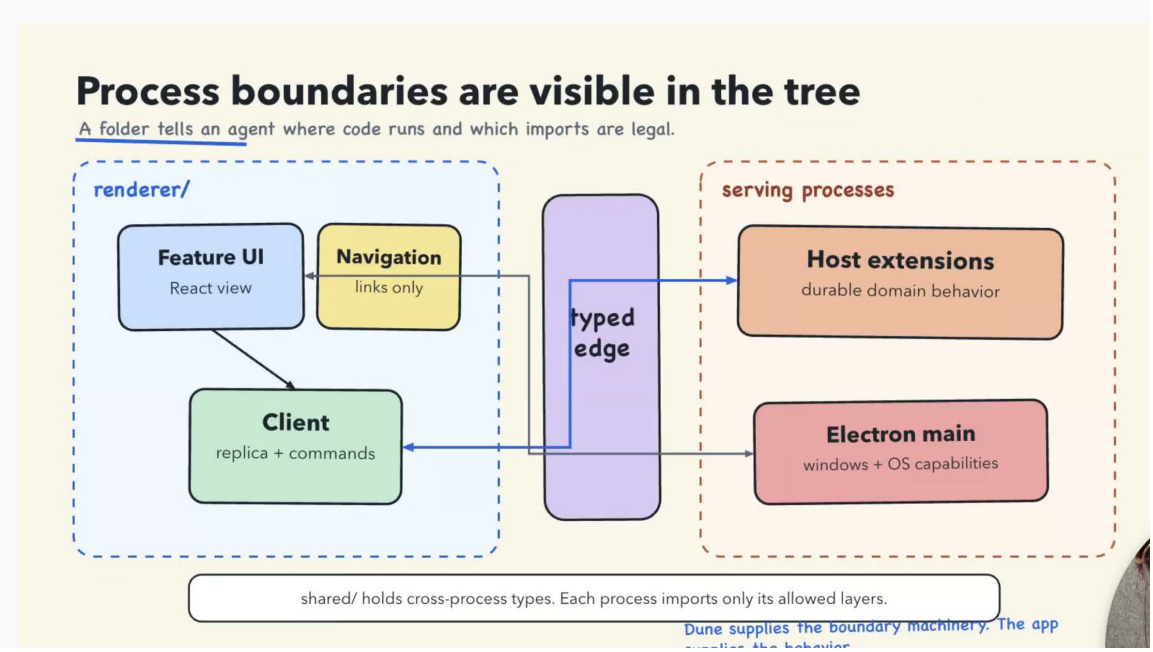


图 6: 渲染侧与服务进程之间经类型化边界通信。[6]

4.3 新增功能沿一条明确的垂直路径前进

更具体的蓝图把一个功能拆成 Feature UI、Client、共享类型边界与 Host extension。React 组件读取命名 hook；Client 持有本地副本与命令；共享边界声明参数与失败类型；Host 承担持久行为。代理修改某个位置时，不需要重新发明进程通信、重试顺序或进程启动逻辑。讲者的判断是：只要遵循这条路径，低上下文代理也较可能保持全局不变量。

[6] 视频画面时间区间：00:28:50–00:29:10。

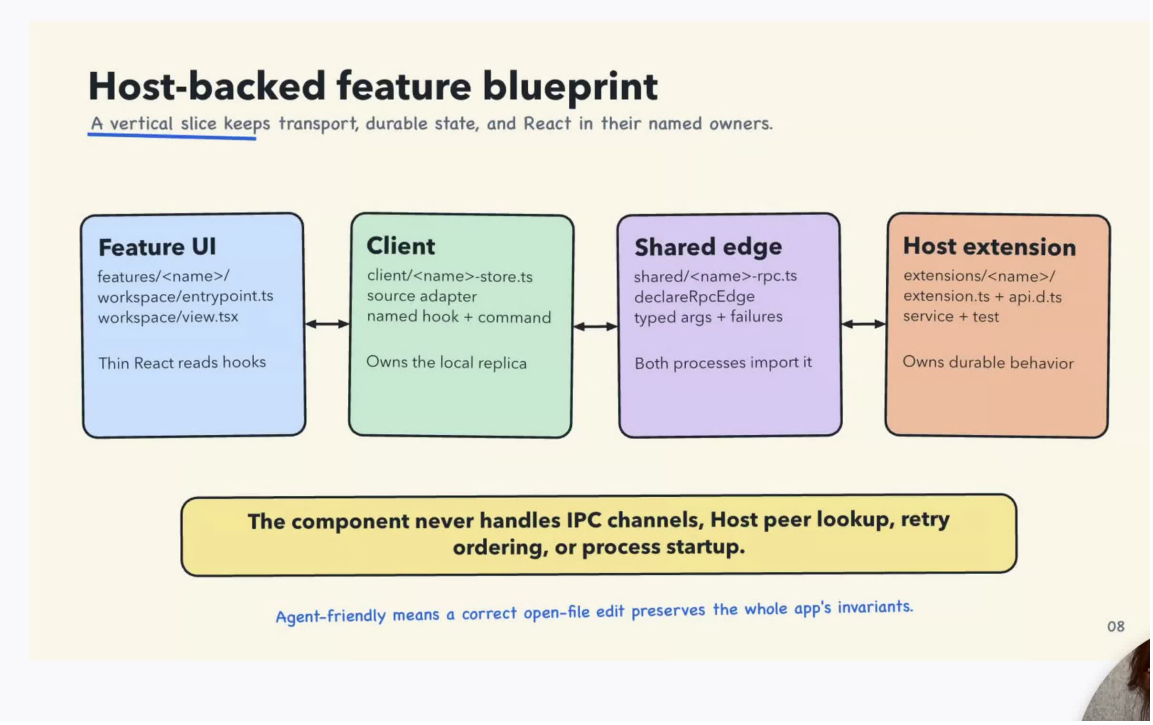


图 7: Dune 的 host-backed 功能蓝图。[7]

“代理友好”的具体含义

明确的目录、少量受祝福的实现路径、可检查的导入边界以及稳定的组件职责，使代理仅凭局部上下文也更容易作出符合系统约束的修改。

4.4 本章小结

Dune 案例把抽象的“代码库是记忆”落实为命名、文件布局、职责分离与依赖约束。可靠性来自可执行的结构，而非期望每个代理都记住所有历史教训。

5 外部事件与内部代理组成自动化闭环

5.1 外层感知触发内层工作

前几章处理的是代理进入代码库后怎样做对。讲者还介绍 Grokbot 的外层循环：接入 Slack、Datadog、Sentry、PlanetScale 等工具，订阅线程与告警，聚合足够的任务上下文，再启动云端代理。Cursor 的自动化或 SDK 可以复用已有技能与规则。讲者强调，不必先建设复杂的“公司大脑”；把真实工具连通并让任务有清晰触发条件，就已能形成有价值的自动化。

两层闭环

外层负责发现事件、收集语境、触发任务；内层负责定位功能、修改代码、运行验证、提交可审查结果。只有内层证据足够可靠，外层自动触发才不会放大噪声。

[7] 视频画面时间区间：00:30:20–00:30:40。

她展示了自动复现 bug 报告、判断问题在当前主分支是否已经解决、自动打开 PR 的示例。截图记录了一个代理报告“在之前提交上复现，修复后的主分支上不再出现”；它说明了工作流形式，不能单独证明所有自动化任务的成功率。

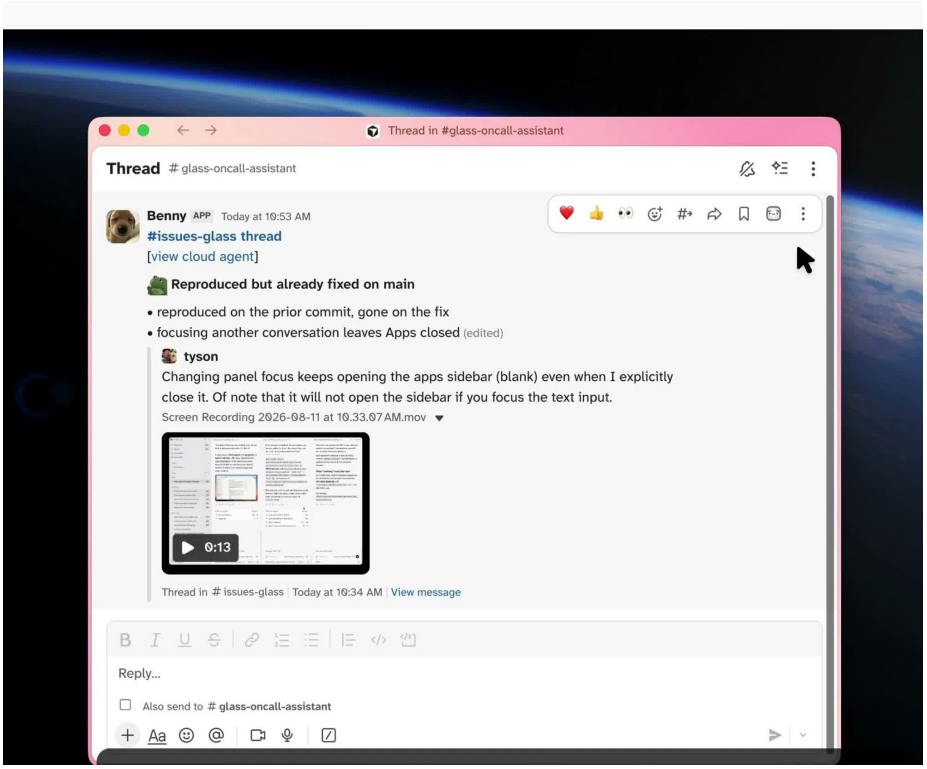


图 8: 从问题报告出发的自动复现示例。[8]

5.2 本章小结

自动化把代理从“人发起的单次会话”扩展为响应产品与运维事件的团队能力。验证证据、代码约束与流程技能是它的基础设施。

6 总结与延伸

6.1 讲者最后强调什么

结尾讲者再次回到“每次纠正代理”的五层清单：优先改造架构、数据结构与代码库，使错误路径消失；再用静态分析阻断复发；再叠加规则、自动审查和技能。她说，高信任环境需要持续的工程投入。一旦环境可靠，代理便能更自由地并行工作，团队中更多成员也能在有限上下文下完成高质量改动。

6.2 把全场压缩成一个工程循环

这场讲座可以凝练为一个循环：观察一次失败；确认它属于功能理解、操作能力、验证证据、质量流程还是架构边界问题；把修正写进相应层；用实际任务检验修正是否生效；再扩大自动化覆盖。其核心单位并非“多开一个代理”，而是“把一次人类纠错转成可复用的系统改进”。

[8] 视频画面时间区间：00:34:50–00:35:10。

落地顺序

选一个重复出现、可复现的真实问题；建立功能地图和稳定的应用控制接口；要求代理提交可复核证据；把高频错误沉淀为代码库约束或静态检查；最后才逐步增加并行与事件触发。

这也留下几个开放问题：验证覆盖怎样量化；何时值得将经验性检查升级为更严格的不变量；怎样防止自动维护的功能地图过时；对不同风险等级的变更，何时仍需要人工把关。讲者没有提供统一阈值，团队应根据自身系统的故障成本、测量能力和审查带宽决定。

6.3 本章小结

“米其林厨房”是对职责分工的提醒：代理可以执行许多工序，团队仍要负责菜单、器具、卫生标准与最终出品。高产量若没有信任机制支撑，只会加快错误传播；可靠环境才让速度有长期价值。